Optimizing windowed arithmetic for quantum attacks against RSA-2048

Alessandro Luongo*†, Varun Narasimhachar‡, Adithya Sireesh§
*Center for Quantum Technologies (CQT), Singapore
†Inveriant Pte. Ltd., Singapore
‡Agency for Science, Technology and Research (A*STAR), Singapore
§Quantum Software Lab, University of Edinburgh, Edinburgh, United Kingdom

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132436

***Abstract*—Windowed arithmetic is a technique for reducing the cost of quantum arithmetic circuits with space–time trade-offs using memory queries to precomputed tables. It can reduce the asymptotic cost of modular exponentiation from $\mathcal{O}(n^2)$ to $\mathcal{O}(n^2/\log^2 n)$ operations, resulting in the current state-of-the-art compilations of quantum attacks against modern cryptography. We introduce several optimizations to windowed arithmetic. Notably, we effect an approximate 50% reduction in the costs of uncomputing memory lookups in quantum factoring applications. We validate our optimizations by improving the gate count of quantum attacks against public-key cryptography by 1.5% to 3.4%, depending on the key size. We also enable a 16% runtime reduction at the cost of a 12% increase in qubit count. Our techniques can be used to reduce the complexity of not only factoring algorithms but also a wide range of quantum algorithms that rely on windowed arithmetic.**

***Index Terms*—modular arithmetic, quantum computing, circuit optimization, qubit, design automation.**

I. Introduction

Efficient quantum arithmetic circuits are fundamental to a wide range of quantum algorithms, from cryptographic applications to simulations in physics and chemistry, machine learning, and finance. For example, these circuits feature prominently in Shor's quantum factoring algorithm [1], whose implementation would render many cryptographic schemes, including RSA and ECC (elliptic curve cryptography), insecure. Despite the theoretical promise of speedups offered by quantum algorithms, practical implementations demand highly optimized arithmetic circuits to minimize qubit and gate resource overheads. At the core of quantum factoring algorithms there is a circuit for the modular exponentiation operation, which dominates the costs—including physical qubit count, gate count, and runtime—required for their execution. Hence, optimizing modular arithmetic subroutines within these algorithms is crucial for making quantum attacks on cryptography practically viable. In quantum computing, more significantly than in the classical case, circuit optimizations—such as reducing the number of gates or the space—can mean the difference between being able to run an algorithm and being unable to execute it at all. In this regard, significant progress has already been made in improving quantum arithmetic algorithms [2]–[6] and their applications, especially in cryptanalysis, e.g., with newer variants of factoring algorithms and more efficient modular arithmetic subroutines to reduce resource overheads [7]–[11]. One notable optimization is windowed quantum arithmetic, introduced by Gidney [12], which reduces the costs of modular multiplication by precomputing lookup tables and merging multiple operations into segments or "windows". This technique improves the scaling of modular exponentiation of n-bit numbers from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^3/\log^2 n)$, significantly reducing the overhead of modular exponentiation. The lookup–addition approach offloads some operations to a classical computer, thus optimizing quantum resource usage. Our work builds on these advancements by introducing four novel optimizations to windowed arithmetic, with a focus on cryptographically relevant instances.

Through this work, we aim to emphasize the importance of enhancing existing subroutines in known quantum algorithms, as these optimizations significantly affect resource estimates for impactful applications.

Main results: In this paper, we study four improvements for windowed arithmetic circuits for modular multiplication and examine their impact on quantum algorithms for breaking RSA-2048. In Section III, we present four key algorithmic optimizations to the windowed arithmetic method, focusing on reducing the Toffoli cost and depth of memory lookups and uncomputation of lookups (unlookups), as well as minimizing the number of required lookups in the modular exponentiation algorithm used by Gidney–Ekerå (GE) [7]. First, we show how the cost of unlookups can be reduced by 50% asymptotically. We also illustrate how certain addresses can be bypassed, significantly reducing both circuit depth and the overall lookup cost. Furthermore, we demonstrate that by merging multiple lookup-addition operations into a single, larger lookup at the start of the modular exponentiation circuit, additional savings in both Toffoli gate count and circuit depth can be achieved.

In Section IV, to further showcase the impact of our optimizations, we study the improved physical qubit count, runtime, and error-correction overhead, providing updated resource estimates for attacking RSA-2048. Concretely, we test our improvements within the GE framework [7] and demonstrate a reduction in the computational volume for factoring RSA-2048 integers. Combined, these optimizations yield reductions in the Toffoli count for attacks against RSA by 1.5% to 3.4%, depending on the key size. These improvements help find parameters for the original GE algorithm that lead to a slight reduction in the overall computational volume for factoring RSA-2048. Specifically, we find that the time required decreases by about 16%, though with a 12% increase in the physical qubit count. The final two optimizations lead to a reduction in the number of *logical* qubits used in the windowing operation; however, we could not translate these reductions into a corresponding decrease in physical qubits or total runtime.

II. Preliminaries and Notation

We denote quantum registers as collections of qubits used for storing quantum information. For example, the state $|x\rangle_n$ refers to an n-qubit register in the computational basis with value $x \in \{0, 1, \ldots, 2^n - 1\}$. For windowed arithmetic, we divide a register $|x\rangle_n$ into a consecutive set of qubits or "windows" of size w qubits, with each window denoted as $x^{(i,w)}$ for $i = 0, 1, \ldots, \lceil n/w \rceil - 1$. Each $x^{(i,w)}$ is treated as an

Corresponding author: asireesh@ed.ac.uk

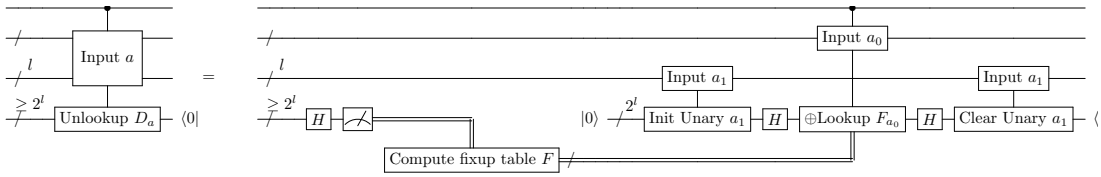

Fig. 1: Uncomputation of lookups i.e. unlookup circuit from [12] on a table of size $L = 2^l$, where l is the number of bits in the address register. Here the unlookup is performed on only half the address space, thus leading to a Tof gate count of $\mathcal{O}(\sqrt{L})$

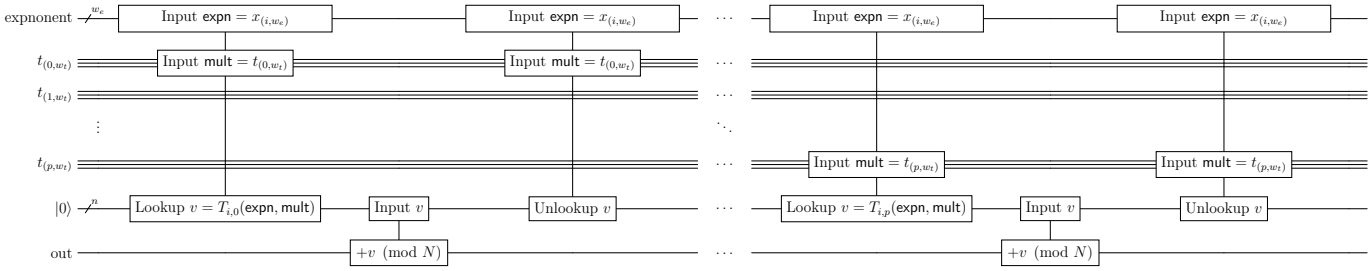

Fig. 2: Windowed modular exponentiation by [12]. Here, we are executing the modular multiplication of $b^{2^{iw_e} x^{(i,w_e)}}$ (the exponentiated value of the of the i^{th} window $x^{(i,w_e)}$ of the exponent), with the register $t = b^{x^{(0,iw_e)}}$ which is currently holding the exponentiation of the first $i-1$ windows of the exponent register.

independent w-bit value. For instance, the overall value of x can be expressed in terms of w-bit windows as follows:

$$x = \sum_{i=0}^{\lceil n/w \rceil - 1} x^{(i,w)} \cdot 2^{i \cdot w}.$$

A. Quantum table lookups

Efficient lookup operations form the basis of the the cost savings in windowed arithmetic. Quantum table lookups are a way to load classical or quantum data into memory [13]. For an address register $|a\rangle_l$, we have an associated lookup table of size $L = 2^l$. Let us assume that each memory element in the lookup table is of m bits, therefore, we need m qubits to store the value of the lookup. A lookup operation can be described as follows:

$$|a\rangle_l |y\rangle_m \xmapsto{\text{Lookup}} |a\rangle_l |y \oplus T_a\rangle_m$$

where T_a is the memory element in the lookup table for index a. A specific architecture for a quantum lookup table, termed a QROM lookup, was introduced in [13], requiring L Tof gates per lookup operation. The circuit for the QROM lookup can be seen in [12, Figure 2].

a) Uncomputation: motivation and improvements.: Uncomputation is often necessary in quantum algorithms to reset ancillary qubits and free up space for reuse. While Bennett's method [14] offers a general approach, it can double resource costs. Recent work [6], [12], [13], [15]–[17] has focused on reducing these overheads. For quantum table lookups, uncomputation (or unlookup) becomes critical to reset registers. Simplifications are possible, as shown by Gidney [12], leveraging a measurement-based uncomputation (MBU) technique. First, the lookup register is measured in the X basis, yielding a classical value s, which represents the measurement outcome. This operation can be represented as follows

$$\sum_a |a\rangle_\ell |T_a\rangle_m \xmapsto{\text{Measure}} \sum_a (-1)^{s \cdot T_a} |a\rangle_\ell |0\rangle_m ,$$

where s induces a phase based on T_a. Since the lookup table is known, the phase can be corrected using the following steps:

1. **Unary Conversion**: Split the address register a into a_{low} and a_{high} of u and $\ell - u$ qubits, respectively. Convert a_{low} into a unary basis:

$$\sum_a (-1)^{s \cdot T_a} |a_{\mathsf{high}}\rangle_{\ell-u} |a_{\mathsf{low}}\rangle_u |2^{a_{\mathsf{low}}}\rangle_{2^u} |0\rangle_{m-2^u} .$$

2. **Phase Correction Lookup**: Construct a simplified phase correction table $F_{a_{\mathsf{high}}}$ where $F(x) = 1$ for indices x satisfying $s \cdot T_{a_{\mathsf{high}} x} = 1$. Apply simultaneous phase corrections using the unary register as the lookup register.

3. **Clear Unary**: Perform an inverse unary operation to reset the unary register:

$$\sum_a |a\rangle_\ell |0\rangle_m .$$

The unlookup cost includes the unary conversion, phase correction lookup, and clearing the unary register. Using a temporary logical-AND, the total cost is $2^u + 2^{\ell-u}$ Tof gates. Optimal performance occurs when $u = \ell/2$, minimizing the cost to $2 \cdot 2^{\ell/2} = 2\sqrt{L}$, achieving a square-root speedup over the original lookup. This efficient unlookup operation is illustrated in fig. 1.

B. Windowed quantum arithmetic

The complexity of modular exponentiation can be reduced from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^3/\log^2 n)$ by leveraging precomputed lookup tables. To prepare the state $\sum_x |x\rangle |b^x \mod N\rangle$, where x is a k-bit register, $b^x \mod N$ is expressed as:

$$b^x \mod N = \prod_{i=0}^{k-1} b^{2^i x_i} \mod N,$$

indicating that modular exponentiation consists of repeated controlled modular multiplications with controls on x_i. For each x_i, a controlled

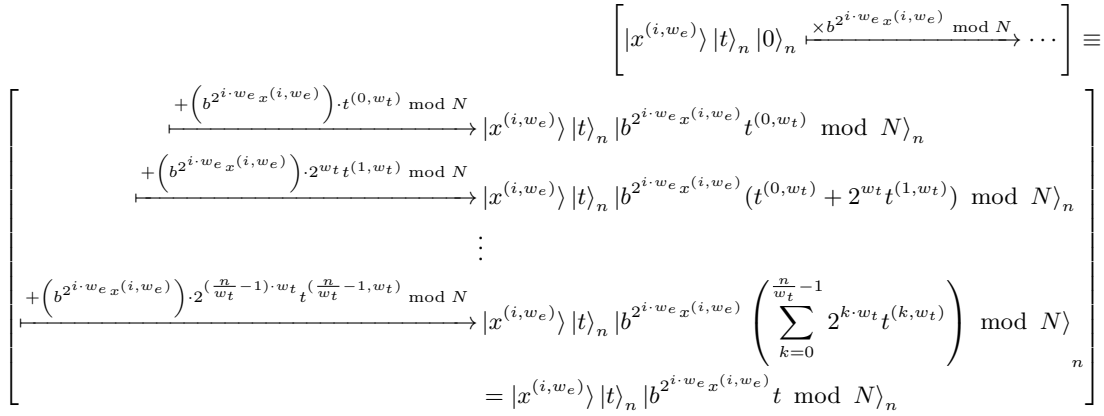

Fig. 3: An illustration of the windowing procedure by Gidney [12] for quantum modular exponentiation. $x^{(i,w_e)}$ represents the i-th window of size w_e in the exponent. $t^{(j,w_t)}$ represents the j-th window of size w_t in the multiplier t.

modular multiplication maps

$$|x_i\rangle_1 |t\rangle_n |0\rangle_n \xmapsto{\times b^{2^i x_i} \bmod N} \begin{cases} |x_i\rangle_1 |t\rangle_n |tb^{2^i} \mod N\rangle_n & \text{if } x_i = 1, \\ |x_i\rangle_1 |t\rangle_n |t\rangle_n & \text{if } x_i = 0. \end{cases}$$

This is further decomposed into modular additions for each bit t_j of t. Iteratively, we compute:

$$|x_i\rangle_1 |t\rangle_n |0\rangle_n \xmapsto{+\left(b^{2^i x_i}\right)\cdot 2^j t_j \bmod N} |x_i\rangle_1 |t\rangle_n |b^{2^i x_i} T \mod N\rangle_n,$$

where $T = \sum_{j=0}^{n} 2^j t_j$. The lookup table, indexed by (x_i, t_j), stores:

t_j	x_i	$(b^{2^i x_i}) \cdot 2^j t_j \mod N$
0	0	0
0	1	0
1	0	2^j
1	1	$2^j b^{2^i} \mod N$

TABLE I: Lookup table entries for a 2 bit lookup-addition based quantum modular multiplication.

This only exponentiates 1 bit of the exponent and has to be repeated k times for the k different bits of the exponent x. However, since all we are doing is performing table lookups followed by additions, we do not need to stop at just looking up 2 bits at a time. Let us assume we take a window w_e of bits for the exponent, and a window w_t of bits for our multiplication register t. This allows us to exponentiate w_e bits at a time as illustrated in figs. 2, 3.

The lookup table is now sized at $L = 2^{w_t + w_e}$ to store every possible combination of w_t and w_e-bit numbers, i.e. lookup value for every pair of t_j and x_i values, which is a product modulo N. Since each lookup and addition operation takes $\mathcal{O}(2^{w_e+w_t} + n)$ operations to retrieve, and we have to exponentiate every window of size w_e to the current window of size w_t, this leads to an asymptotic scaling of $\mathcal{O}(n^3/\log^2 n)$ if we take windows of size $w_e = w_t = \frac{1}{2}\log n$.

III. Improvements

In this section, we present several optimizations for windowed arithmetic circuits, particularly in scenarios involving multiple consecutive lookup operations. These optimizations focus on minimizing the number of Toffoli gates and circuit depth.

A. Deferred uncomputation for windowing circuits

While unlookups are much cheaper than their corresponding lookups, they can be made even cheaper in the multi-lookup setting (i.e. when more than one QROM lookup is involved) if part or all of the address register remains unchanged over many consecutive QROM lookups. This is the exact scenario in the case of the GE factoring algorithm, where for all the lookup-additions involved in multiplying $b^{x^{(i,w_e)}2^{iw_e}}$ (with index i, and window size w_e), the exponent window remains unchanged. These lookups have address $a = x^{(i,w_e)}||t^{(j,w_m)}$ with $j \in \left\{0, 1, \cdots, \frac{n}{w_m} - 1\right\}$, where $t^{(j,w_m)}$ is a window over the multiplication qubits. Each unlookup is of the form:

$$\begin{aligned} &\xmapsto{\mathsf{H,Measure}} (-1)^{S_j \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |S_j\rangle_n |r\rangle_n \\ &\xmapsto{\mathsf{Reset}\ S_j} (-1)^{S_j \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |0\rangle_n |r\rangle_n \\ &\xmapsto{\mathsf{Unary}} (-1)^{S_j \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n \\ &\qquad |\mathsf{unary}(x^{(i,w_e)})\rangle_{2^{w_e}} |r\rangle_n |0\rangle_{n-2^{w_e}} \\ &\xmapsto{\mathsf{Lookup}\ F_{S_j}} |x^{(i,w_e)}\rangle |t\rangle_n \\ &\qquad |\mathsf{unary}(x^{(i,w_e)})\rangle_{2^{w_e}} |r\rangle_n |0\rangle_{n-2^{w_e}} \\ &\xmapsto{\mathsf{Clear\ Unary}} |t\rangle_n |0\rangle_n |r\rangle_n \end{aligned}$$

Each lookup is addressed by $a = \mathsf{exp}||\mathsf{mul}$ that combines an (outer loop) exponentiation window index exp and an (inner loop) addition window index mul. Gidney's [12] unary unlookup reduction is then applied to this construction.

We improve this further, exploiting the nested loop structure. Notice that the outer index exp stays fixed while the circuit iterates over the inner index mul. In our proposed optimization, we use mul as the address of the fixup table F_{S_j} and the unarized exp as the target. The binary-to-unary initialization of exp needs to be done only once before entering the inner loop, and the erasure of the unary register only once right at the end of the inner loop—whereas, in the unmodified version, there is a reset and initialization after every iteration within the inner loop. Thus, the cost of our circuit is dominated by the phase lookups (F_{S_j}) within each iteration, the one-time unary initialization and reset contributing negligibly in comparison. Overall, this leads to a halving of the Toffoli cost of unlookups in the GE factoring algorithm. After every lookup in the inner loop, let's instead perform

the operation (as shown in fig. 4)

$$\xmapsto{\text{H,Measure}} (-1)^{s_j \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |S_j\rangle_n |r\rangle_n$$
$$\xmapsto{\text{Reset } S_j} (-1)^{S_j \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |0\rangle_n |r\rangle_n$$

The value S_j is recorded classically, and a phase fixup table F_{S_j} is constructed (indexed/addressed by mul). At the end of the inner loop, we have $p = \left(\frac{n}{w_t} - 1\right)$ classical bit strings S_j, and p different phase fixup tables F_{S_j} ; $\forall j \in \{0, 1, \cdots, p-1\}$. The state at this stage is:

$$(-1)^{\sum_{k=0}^{p-1} S_k \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |0\rangle_n |r\rangle_n .$$

We can now construct a unary register with input $x^{(i,w_e)}$, and window over the register t to perform our phase corrections with F_{S_j}, i.e.

$$\xmapsto{\text{Lookup } F_{S_0}} (-1)^{\sum_{k=1}^{p-1} S_k \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |\mathsf{unary}(x^{(i,w_e)})\rangle_{2^{w_e}} |r\rangle_n |0\rangle_{n-2^{w_e}}$$

$$\xmapsto{\text{Lookup } F_{S_1}} (-1)^{\sum_{k=2}^{p-1} S_k \cdot \ell_{i,j}} |x^{(i,w_e)}\rangle |t\rangle_n |\mathsf{unary}(x^{(i,w_e)})\rangle_{2^{w_e}} |r\rangle_n |0\rangle_{n-2^{w_e}}$$

$$\vdots$$

$$\xmapsto{\text{Lookup } F_{S_{p-2}}} (-1)^{S_{p-1} \cdot \ell_{i,p-1}} |x^{(i,w_e)}\rangle |t\rangle_n |\mathsf{unary}(x^{(i,w_e)})\rangle_{2^{w_e}} |r\rangle_n |0\rangle_{n-2^{w_e}}$$

$$\xmapsto{\text{Lookup } F_{S_{p-1}}} |x^{(i,w_e)}\rangle |t\rangle_n |\mathsf{unary}(x^{(i,w_e)})\rangle_{2^{w_e}} |r\rangle_n |0\rangle_{n-2^{w_e}}$$

We now clear the unary register to retrieve our phase-free superposition

$$\xmapsto{\text{Clear Unary}} |x^{(i,w_e)}\rangle |t\rangle_n |0\rangle_n |r\rangle_n$$

B. Selective lookups

Although lookup tables differ for different moduli N and different exponent bases b, they exhibit similar structures for certain addresses. During a lookup addition of the i^{th} exponent window (of size w_e) and the j^{th} multiplication window (of size w_m), we construct a classical table $T_{i,j}$ corresponding to addresses stored in the register $m^{(j,w_m)}||x^{(i,w_e)}$. For a classical bit string address $a = \mathsf{mul}||\mathsf{exp}$, with $\mathsf{mul} \in \{0, 1, \cdots, 2^{w_m} - 1\}$ and $\mathsf{exp} \in \{0, 1, \cdots, 2^{w_e} - 1\}$, the lookup table $T_{i,j}$ in question holds the following values:

$$T_{i,j}(\mathsf{mul}, \mathsf{exp}) := (b^{\mathsf{exp}2^{iw_e}} 2^{jw_m} \mathsf{mul}) \mod N.$$

$T_{i,j}$ holds $2^{w_e+w_m}$ different values (for all combinations of length-w_e and -w_m bit strings). Inspecting the values associated with addresses where $\mathsf{mul} = 0$, we see that $T_{i,j}(\mathsf{mul}, *) = 0$ for all values of exp, regardless of the indices i and j. Therefore, we can start the lookups directly from $\mathsf{mul} = 1$ as we know that there is nothing to lookup for $\mathsf{mul} = 0$. This would save us 2^{w_m} lookups per query. On the other hand, for all addresses with $\mathsf{exp} = 0$, we get $T_{i,j}(\mathsf{mul}, \mathsf{exp}) = \mathsf{mul}2^{jw_m}$. Therefore, mul just gets copied directly to the target register. We propose to perform this copying at the beginning of every lookup. Note, the lookup table has to be altered now with lookup values

$$T'_{i,j}(\mathsf{mul}, \mathsf{exp}) = (b^{\mathsf{exp}2^{iw_e}} 2^{jw_m} \mathsf{mul} \mod N) \oplus 2^{jw_m} \mathsf{mul}.$$

We need to alter the table because the initial copying operation is an unconditional operation acting on all address inputs. We need to correct the lookup values where $\mathsf{exp} \neq 0$. The initial copying operation is analogous to the lookup $v = 2^{jw_m} \mathsf{mul}$. Using the corrected table, we find we now lookup $T'_{i,j}(\mathsf{mul}, \mathsf{exp})$, which alters the lookup register to

$$\begin{aligned} v &= 2^{jw_m} \mathsf{mul} \oplus T'_{i,j}(\mathsf{mul}, \mathsf{exp}) \\ &= 2^{jw_m} \mathsf{mul} \oplus (b^{\mathsf{exp}2^{iw_e}} 2^{jw_m} \mathsf{mul} \mod N) \oplus 2^{jw_m} \mathsf{mul} \\ &= (b^{\mathsf{exp}2^{iw_e}} 2^{jw_m} \mathsf{mul}) \mod N \\ &= T_{i,j}(\mathsf{mul}, \mathsf{exp}) \end{aligned}$$

This modification saves us $2^{w_m} - 1$ lookups (minus 1 because for the case $\mathsf{mul} = 0$, else we would be double counting the savings from the previous optimization). Refer to the circuit shown in fig. 5 for this copy-and-update-table lookup operation.

C. Larger initial lookup

While windowed modular exponentiation typically processes the exponent in small windows of size $\frac{1}{2} \log n$ to balance precomputation (i.e. size of lookups) and the number of iterations of the lookup-additions, we investigate an optimization for the initial phase of the algorithm. By combining multiple initial exponent windows into a single, larger lookup operation, we show below how to slightly reduce the number of modular multiplications and thus decrease the overall Toffoli gate count. Recall that with $x^{(i,w_e)}$ we represent the i-th window of size w_e in the exponent and with $t^{(k,w_t)}$ we represent the k-th window of size w_t in the multiplication register. We also recall from Section II-B how modular exponentiation is performed using windowing:

$$|x^{(0,w_e)}\rangle_{w_e} |1\rangle_n |0\rangle_{2n}$$
$$\xmapsto{\times b^{x^{(0,w_e)}} \mod N} |x^{(0,w_e)}\rangle |b^{x^{(0,w_e)}}\rangle_n |0\rangle_{2n}$$
$$\xmapsto{\times b^{2^{w_e} x^{(1,w_e)}} \mod N} |b^{x^{(0,2w_e)}}\rangle_n |0\rangle_{2n}$$
$$\vdots \text{ Iterate over } i \text{ exponent windows}$$
$$\xmapsto{\times b^{2^{i \cdot w_e} x^{(i,w_e)}} \mod N} |b^{x^{(0,(i+1)w_e)}}\rangle_n |0\rangle_n$$

The first modular multiplication in the circuit can be reduced to a single lookup operation over the first $(i+1)w_e$ bits of the exponent. However, as i increases, this approach becomes more expensive than regular windowed modular multiplication, potentially negating the benefits of windowed arithmetic due to exponential growth in lookup table size. We explore the tradeoff between a direct lookup exponentiation for the first e bits of the exponent versus the cost of a lookup-addition based exponentiation.

D. Sliced windowing

During the windowing stage of the GE factoring algorithm, precomputed lookup table values are loaded into an n-qubit lookup register. We propose a method to reduce the number of logical qubits required for these lookups by effectively utilizing carry runways [18]. The GE algorithm employs carry runways to parallelize quantum addition. Assuming the carry runways divide the n-qubit quantum register into m pieces, each piece is of size $\frac{n}{m}$. Our approach involves loading only $\frac{n}{m}$ or a multiple of $\frac{n}{m}$ bits of the lookup table at a time. We then add this loaded register of size $\frac{n}{m}$ to the relevant quantum register, uncompute the values, and load the next $\frac{n}{m}$ bits. This strategy reduces the abstract qubit count of the GE factoring algorithm from $3n$ to $2n + \frac{n}{m}$. Although this increases the Toffoli count of the lookups by a factor of m, it can be managed by rearranging the order of the lookups and additions. We propose two variants of the sliced windowing scheme: Variant A, which reduces the logical qubit count

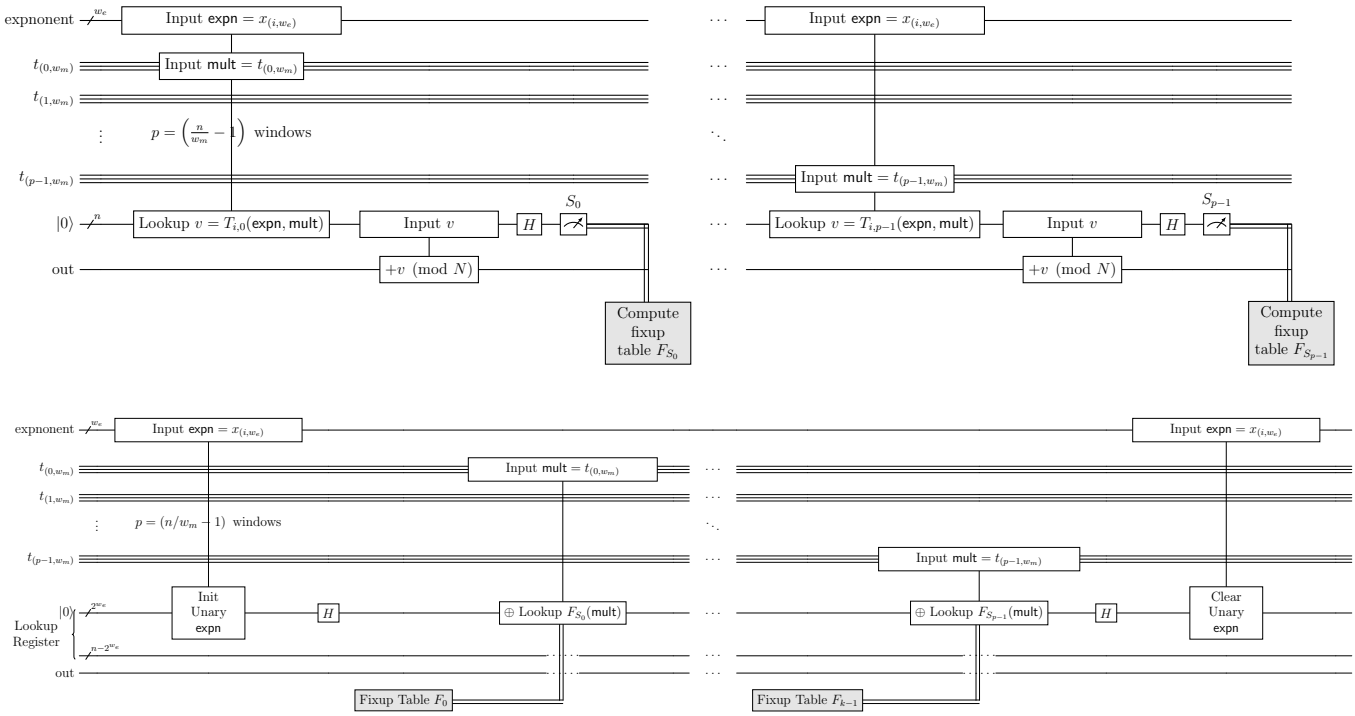

Fig. 4: Our proposed circuit for deferred uncomputation. The lookup additions have 2 stages. In stage one, the windowed multiplication with lookup qubit reset and no phase correction (top) is performed, followed stage 2 where the circuit for deferred phased correction (bottom) is executed. Notice that the unary conversion is performed only one, and on the exponent window as it is common for all lookups and can be reused. This is different from fig. 2 where the unlookups/phase corrections are performed after every single lookup addition.

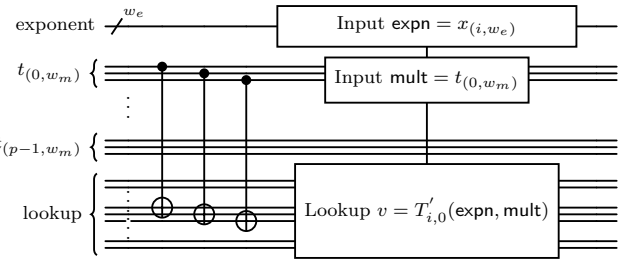

Fig. 5: Proposed optimization for pruned table. The CNOTs before the table lookup perform a copy operation, followed by an altered table lookup.

without impacting the Toffoli count, and Variant B, which reintroduces ancillas to lower the Toffoli count at the cost of increased circuit depth.

Variant A: Logical qubit reduction with sliced windowing: In this variant, lookup additions are performed on the first half of the target register $t[0 : n/2]$, while the second half $t[n/2 : n]$ serves as ancillas for Gidney's logical-AND adder. At each step, we only lookup the first $n/2$ bits into memory, compared to the full n bits in [12]. Each lookup costs $2^{w_e+w_m} = 2^{2\lfloor \frac{1}{2} \log n \rfloor}$ in Toffoli gates and depth. The addition of $n/4$-sized register pieces costs $n/2$ Toffoli gates, with depth n. In the second phase, another lookup-addition is performed with Cuccaro RCA addition (no ancillas), costing n Toffoli gates and depth n. Overall, this protocol does not reduce the Toffoli count but achieves a logical qubit reduction of $n/2$, with an increase in depth by n.

Variant B: Reintroducing temporarily protected ancillas for a lower Toffoli count: This variant reduces the Toffoli count by reusing ancillas that are idle during part of the computation. Instead of loading the entire lookup table, we process half at a time and repurpose ancillas saved in Variant A for addition into the second half of the target register. In Variant A, Gidney's adder [6] was used for the first half of the target register, with the second half acting as ancillas. For the second half, we switched to Cuccaro's ripple-carry adder due to the lack of ancillas. Here, we reintroduce the ancillas saved during the first stage, protecting them temporarily and "borrowing" them for Gidney's adder into the second half. This approach reduces the Toffoli count by $n/2$ while maintaining the same logical qubit count. However, it may increase circuit depth, as the ancillas must remain protected for longer, potentially offsetting the benefits in certain configurations.

IV. Application to breaking RSA-2048

In this section, we build upon the resource estimation framework introduced by Gidney and Ekerå [7] by incorporating our proposed enhancements to windowed arithmetic. Our analysis targets a superconducting qubit architecture, with computations encoded in rotated surface codes and logical operations performed using lattice surgery [19], [20]. We assume a physical error rate of 10^{-3} (and also test with a more optimistic 10^{-4} error rate), reflecting the fidelity of operations such as single- and two-qubit gates, state initialization, and measurements. Additionally, we consider a cycle time of 1μs, representing the duration required to measure all stabilizer of a surface code patch. These parameters are chosen to align with anticipated capabilities of future fault-tolerant quantum hardware and provide a consistent baseline for comparison. For a distance-d rotated surface code, we require $2(d+1)^2$ physical qubits to encode

n	n_e	**gate**$_{err}$	L_1	L_2	dev_{off}	g_{mul}	g_{exp}	g_{sep}	%	v.p.r	$\mathbb{E}$[vol]	Mqb	hrs	$\mathbb{E}$[hrs]	B Tofs
1024	1493	10^{-3}	15	27	5	5	5	1024	6%	0.507	0.539	9.624	1.264	1.344	0.407
2048	3029	10^{-3}	15	27	4	5	5	1024	31%	4.047	5.865	19.249	5.046	7.313	2.698
3072	4565	10^{-3}	17	29	6	4	5	1024	9%	18.328	20.141	37.897	11.607	12.755	9.885
4096	6101	10^{-3}	17	31	9	4	5	1024	5%	47.963	50.488	54.616	21.077	22.186	23.038
1024	1493	10^{-4}	7	13	4	5	5	512	5%	0.07	0.073	2.637	0.633	0.667	0.415
2048	3029	10^{-4}	7	13	4	5	5	512	21%	0.558	0.706	5.273	2.538	3.212	2.774
3072	4565	10^{-4}	9	15	5	5	5	768	2%	2.92	2.98	9.12	7.686	7.843	8.595
4096	6101	10^{-4}	9	15	5	4	5	512	3%	6.791	7.001	15.241	10.693	11.024	23.773
n	n_e	**gate**$_{err}$	L_1	L_2	dev_{off}	g_{mul}	g_{exp}	g_{sep}	%	v.p.r	$\mathbb{E}$[vol]	Mqb	hrs	$\mathbb{E}$[hrs]	B Tofs
1024	1493	10^{-3}	15	27	4	5	5	1024	6%	0.488	0.519	9.624	1.217	1.295	0.393
2048	3029	10^{-3}	17	27	6	5	5	1024	20%	4.419	5.524	21.616	4.906	6.133	2.656
3072	4565	10^{-3}	17	29	4	4	5	1024	9%	17.727	19.48	37.897	11.226	12.336	9.742
4096	6101	10^{-3}	17	31	8	4	5	1024	5%	46.424	48.867	54.616	20.4	21.474	22.8
1024	1493	10^{-4}	7	13	4	5	5	512	5%	0.067	0.071	2.637	0.612	0.644	0.402
2048	3029	10^{-4}	7	13	3	5	5	512	21%	0.541	0.684	5.273	2.461	3.115	2.72
3072	4565	10^{-4}	9	15	5	5	5	768	2%	2.851	2.909	9.12	7.503	7.656	8.486
4096	6101	10^{-4}	9	15	5	4	5	512	3%	6.588	6.792	15.241	10.374	10.695	23.559

TABLE II: Comparison of resource estimates for the GE factoring algorithm without improvements (top) and with our proposed optimizations (bottom). The columns are: the number of bits of the number to factor (n), the bits in the exponentiation register (n_e), the gate error rate ($gate_{err}$, e.g., 10^{-3}), the distances for the first and second layers of error correction (L_1 and L_2), the deviation due to padding error in coset representation (dev_{off}), the window size for the multiplication register in windowed arithmetic (g_{mul}), the window size for the exponentiation register in windowed arithmetic (g_{exp}), the separation in oblivious ripple carry adders (g_{sep}), the retry risk (probability of error during computation, as a percentage), the computation volume for a single run expressed in megaqubit-days (v.p.r.), the expected computation volume accounting for retry risk ($\mathbb{E}$[volume]), the number of logical qubits in millions (megaqubits, Mqb), the runtime for a single run in hours (Runtime), the expected runtime accounting for errors ($\mathbb{E}$[hours]), and the total number of Toffoli gates in billions (B Tofs)

a single logical qubit of information. A code distance of $d = 27$ is proposed as sufficient for factoring RSA-2048 on large-scale quantum hardware, balancing the trade-off between logical error rates and resource requirements. Our goal is to minimize the *skewed volume*, a key metric that balances the trade-offs between qubit resources and runtime, defined as:

$$\text{skewed\_volume} = (\text{Mqb})^{1.2} \times \mathbb{E}[\text{hrs}]$$

where Mqb is the total number of physical qubits in megaqubits (millions of qubits), and $\mathbb{E}$[hrs] is the expected runtime in hours, accounting for the likelihood of retries due to logical errors, with the exponent of 1.2 reflecting a preference for reducing qubit (Mqb) over runtime ($\mathbb{E}$[hrs]), and can be varied based on the specific use cases.

The improvements described in Sections III-A, III-B, and III-C focus on reducing the Toffoli count and computational volume for modular arithmetic operations. When combined, these optimizations yield reductions in the Toffoli count and depth for cryptographically relevant attacks on RSA factoring by 1.5% to 3.4%, as summarized in Table II. More specifically, we also see nearly a 16% reduction in the expected runtime of the algorithm for factoring RSA-2048 bit integers at the cost of a 12% increase in the physical qubit count (owing to the reduced retry risk of the algorithm). The sliced windowing technique, as described in Section III-D, offers a trade-off at a logical level. While it reduces the number of logical qubits required, it comes at the expense of an increased circuit depth. This added depth necessitates higher-distance QEC codes to ensure fault tolerance, which can lead to a net increase in the overall resource cost, and hence is not included in our comparison. Whether sliced windowing can be used to reduce the number of physical qubits for a particular kind of hardware architecture and an error correction code is a non-trivial question, which is left for future research.

The techniques presented in this work, while demonstrated through their application to improve the efficiency of factoring algorithms, have broader utility. They can be applied to simplify and optimize algorithms that rely on (windowed) arithmetic, thereby improving both performance and scalability across a wider range of computations.

ACKNOWLEDGEMENTS

This work started when AS was working at Inveriant Pte. Ltd. AS is supported by Innovate UK under grant *10004359*. This work is supported by the National Research Foundation, Singapore, and A*STAR under its Centre for Quantum Technologies Funding Initiative (S24Q2d0009). We also acknowledge funding from the Quantum Engineering Programme (QEP 2.0) under grants *NRF2021-QEP2-02-P05* and *NRF2021-QEP2-02-P01*. We thank Anupam Chattopadhyay for useful discussions on quantum arithmetic. We thank Filippo Miatto, Ilan Tzitrin, and Rafael Alexander for useful discussions on resource estimations on photonic architectures, and Miklos Santha useful discussions on space-time trade-offs in arithmetic.

REFERENCES

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.

[2] T. G. Draper, "Addition on a quantum computer," *arXiv preprint quant-ph/0008033*, 2000.

[3] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, "A new quantum ripple-carry addition circuit," *arXiv preprint quant-ph/0410184*, 2004.

[4] G. D. Kahanamoku-Meyer and N. Y. Yao, "Fast quantum integer multiplication with zero ancillas," *arXiv preprint arXiv:2403.18006*, 2024.

[5] D. Litinski, "Quantum schoolbook multiplication with fewer toffoli gates," *arXiv preprint arXiv:2410.00899*, 2024.

[6] C. Gidney, "Halving the cost of quantum addition," *Quantum*, vol. 2, p. 74, 2018.

[7] C. Gidney and M. Ekerå, "How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[8] D. Litinski, "How to compute a 256-bit elliptic curve private key with only 50 million toffoli gates," *arXiv preprint arXiv:2306.08585*, 2023.

[9] É. Gouzien, D. Ruiz, F.-M. Le Régent, J. Guillaud, and N. Sangouard, "Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126 133 cat qubits," *Physical Review Letters*, vol. 131, no. 4, p. 040602, 2023.

[10] C. Chevignard, P.-A. Fouque, and A. Schrottenloher, "Reducing the number of qubits in quantum factoring," *Cryptology ePrint Archive*, 2024.
[11] S. Wang, X. Li, W. J. B. Lee, S. Deb, E. Lim, and A. Chattopadhyay, "A comprehensive study of quantum arithmetic circuits," *arXiv preprint arXiv:2406.03867*, 2024.
[12] C. Gidney, "Windowed quantum arithmetic," *arXiv preprint arXiv:1905.07682*, 2019.
[13] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Paler, A. Fowler, and H. Neven, "Encoding electronic spectra in quantum circuits with linear t complexity," *Physical Review X*, vol. 8, no. 4, p. 041015, 2018.
[14] C. H. Bennett, "Logical reversibility of computation," *IBM Journal of R&D*, vol. 17, no. 6, pp. 525–532, 1973.
[15] C. Jones, "Low-overhead constructions for the fault-tolerant toffoli gate," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 87, no. 2, p. 022328, 2013.
[16] N. Kornerup, J. Sadun, and D. Soloveichik, "Tight bounds on the spooky pebble game: Recycling qubits with measurements," *arXiv preprint arXiv:2110.08973*, 2021.
[17] A. Luongo, A. M. Miti, V. Narasimhachar, and A. Sireesh, "Measurement-based uncomputation of quantum circuits for modular arithmetic," *arXiv preprint arXiv:2407.20167*, 2024.
[18] C. Gidney, "Approximate encoded permutations and piecewise quantum adders," *arXiv preprint arXiv:1905.08488*, 2019.
[19] D. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, "Surface code quantum computing by lattice surgery," *New Journal of Physics*, vol. 14, no. 12, p. 123011, 2012.
[20] A. G. Fowler and C. Gidney, "Low overhead quantum computation using lattice surgery," *arXiv preprint arXiv:1808.06709*, 2018.